Unsupervised Learning: Clustering Project

Spotify Audio Clustering: Unsupervised Learning on 32.833 Songs from 6 Genres

by Monica Gayatri K.

1. Main Objective

The dataset of Spotify Audio contains detailed information regarding the name of the song, the audio features metadata, and the acoustic Attributes that were released from 1957 to 2020.

The dataset comprises 32,833 **instances and 23 Features**. With the information of the dataset, we can create **a clustering of a grouped playlist from all the listed songs using various unsupervised learning methods based on our finding from the dataset EDA.** We'll also use **Silhouette Score (Internal Structural Health), Davies-Bouldin Index, and Calinkski Harabasz** to decide which model is the most optimal. Below is the flow diagram process for the this project:

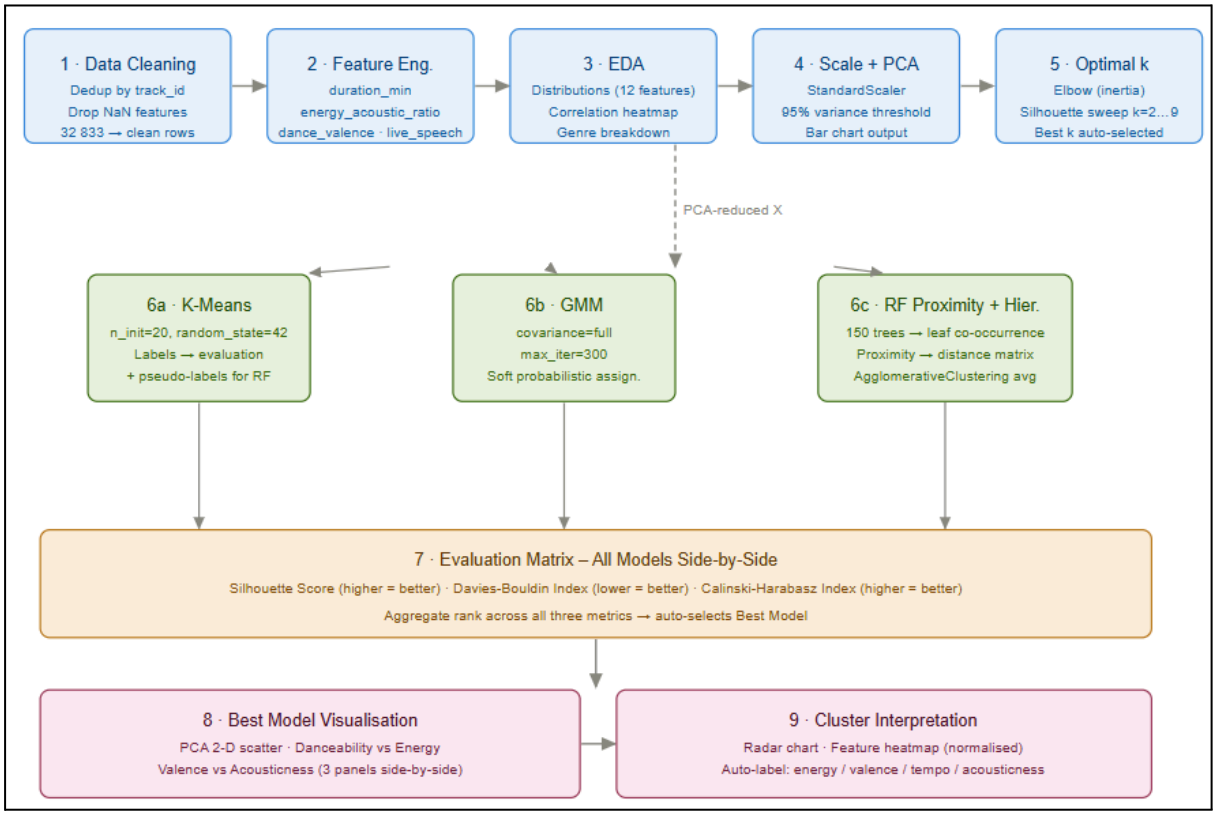


Fig.1. Flow diagram process of unsupervised learning: clustering project.

The clustering results will reveal distinct patterns regarding physical and agronomic traits. These patterns can subsequently be utilized to facilitate the identification of specific, desirable plant profiles.

2. Exploration Data, Data Cleaning, and Feature Engineer

2.1 Exploration Data

2.1.1 Gathering Information

As the dataset contains 23 variables/Features, we need to get a snippet of the dataset first through data.head() to get a snippet of the dataset of each listed song in Fig.2, then use *data.info*() in Fig. 3. to understand the datatype, total entry, and NaN existence.

	track_id	track_name	track_artist	track_popularity	track_album_id	track_album_name	track_album_release_date	playlist_name	playlist_id	playlist_genre	...	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	duration_ms
0	6f807x0ima9a1j3VPbc7VN	I Don't Care (with Justin Bieber) - Loud Luxur...	Ed Sheeran	66	2oCs0DGTsRO98Gh5ZSl2Cx	I Don't Care (with Justin Bieber) [Loud Luxury...	2019-06-14	Pop Remix	37i9dQZF1DXcZDD7cfEKhW	pop	...	6	-2.634	1	0.0583	0.1020	0.000000	0.0653	0.518	122.036	194754
1	0r7CVbZTWZgbTCYdfa2P31	Memories - Dillon Francis Remix	Maroon 5	67	63rPSO264uRjW1X5E6cWv6	Memories (Dillon Francis Remix)	2019-12-13	Pop Remix	37i9dQZF1DXcZDD7cfEKhW	pop	...	11	-4.969	1	0.0373	0.0724	0.004210	0.3570	0.693	99.972	162600
2	1z1Hg7Vb0AhHDiEmnDE79l	All the Time - Don Diablo Remix	Zara Larsson	70	1Ho5mj2eLcsrR0vE9gThr4	All the Time (Don Diablo Remix)	2019-07-05	Pop Remix	37i9dQZF1DXcZDD7cfEKhW	pop	...	1	-3.432	0	0.0742	0.0794	0.000023	0.1100	0.613	124.008	176616
3	75FpbthrwQmzHlBJLuGdC7	Call You Mine - Keanu Silva Remix	The Chainsmokers	60	1nqYsOef1yKKuGOVchbsk6	Call You Mine - The Remixes	2019-07-19	Pop Remix	37i9dQZF1DXcZDD7cfEKhW	pop	...	7	-3.778	1	0.1020	0.0287	0.000009	0.2040	0.277	121.956	169093
4	1e8PAfcKUYoKkxPhrHqw4x	Someone You Loved - Future Humans Remix	Lewis Capaldi	69	7m7vv9wlQ4i0LFuJiE2zsQ	Someone You Loved (Future Humans Remix)	2019-03-05	Pop Remix	37i9dQZF1DXcZDD7cfEKhW	pop	...	1	-4.672	1	0.0359	0.0803	0.000000	0.0833	0.725	123.976	189052

Fig.2. data.head()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32833 entries, 0 to 32832
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   track_id                  32833 non-null  object
 1   track_name                32828 non-null  object
 2   track_artist              32828 non-null  object
 3   track_popularity          32833 non-null  int64
 4   track_album_id            32833 non-null  object
 5   track_album_name          32828 non-null  object
 6   track_album_release_date  32833 non-null  object
 7   playlist_name             32833 non-null  object
 8   playlist_id               32833 non-null  object
 9   playlist_genre            32833 non-null  object
 10  playlist_subgenre         32833 non-null  object
 11  danceability              32833 non-null  float64
 12  energy                    32833 non-null  float64
 13  key                       32833 non-null  int64
 14  loudness                  32833 non-null  float64
 15  mode                      32833 non-null  int64
 16  speechiness               32833 non-null  float64
 17  acousticness              32833 non-null  float64
 18  instrumentalness          32833 non-null  float64
 19  liveness                  32833 non-null  float64
 20  valence                   32833 non-null  float64
 21  tempo                     32833 non-null  float64
 22  duration_ms               32833 non-null  int64
dtypes: float64(9), int64(4), object(10)
memory usage: 5.8+ MB
```

Fig.3. data.info().

2.1.2 Insight

Based on Fig. 2. And Fig. 3., the dataset has:

1. From Fig. 3. data.info(), there are a total 23 columns, which entails 23 columns of features.
2. Based on the dataset's snippet Fig.3. Almost all the values are in number and all the data types are mostly showing integer or float. Except, a few Features that have an object datatype. We'll need to change the data type into integer in Feature engineer.
3. There is some missing value from each column.

2.2 Data Cleaning

As the checking in Fig.2., we found a few columns that had missing values and we need to replace it with the modes or median. In the data cleaning we'll check if there's any duplicated values in the data set by using *len(data.duplicated())* to get the total count of non-duplicated data. Turns out, the dataset doesn't have any duplicated data because the output is the same as the dataset total entries.

2.3 Feature Engineer

2.3.1 Gathering Information

In Feature engineering, we'll create new columns as referred to in Fig.4. that will help the models find better patterns.

```
Engineered features : ['duration_min', 'energy_acoustic_ratio', 'dance_valence', 'live_speech']
```

Fig.4. data.info().

3. Data Visualization

In this section we'll check the histogram of distribution, the correlation heatmap, and the PCA Bar chart of the cleaned dataset to get a better understanding of the dataset.

3.1 The Histogram of Distribution

In order to get a better understanding of the base features patterns, we'll use the Scatter Plot Matrix, in Fig.5, to see the shape of each feature across all songs. From the graph we can see columns 'energy' and 'danceability' tend to be normally distributed, which indicates most songs are moderate, while columns 'instrumentalness' and 'speechiness' are heavily skewed, which indicates most songs have very little of either. In addition, column 'loudness' clusters around −5 to −10 dB for most tracks.

```
base_features = [
    "danceability", "energy", "loudness", "speechiness",
    "acousticness", "instrumentalness", "liveness", "valence",
    "tempo", "duration_min", "energy_acoustic_ratio", "dance_valence"
]
```

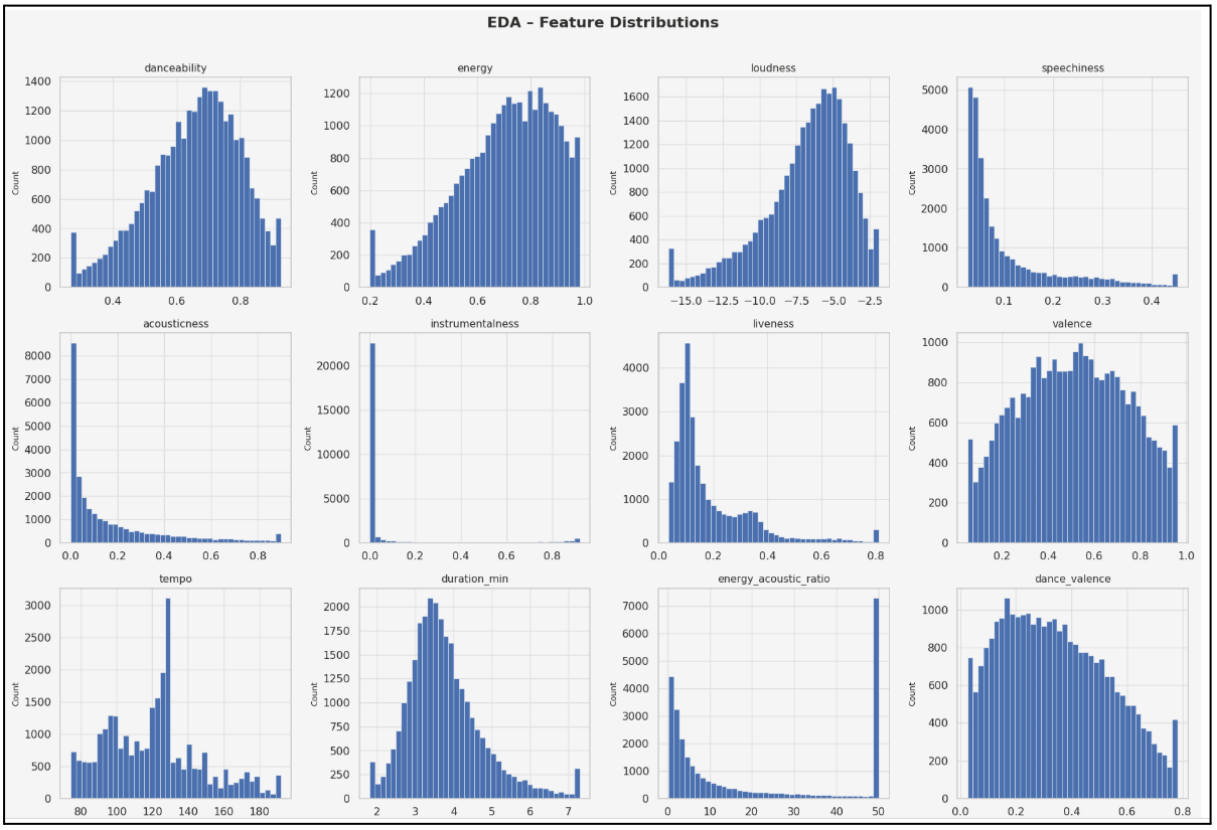


Fig.5. Feature Distributions.

3.2 The Correlation Heatmap

The correlation heatmap shows which features have multicollinearity/redundancy and how strong their correlation is in Fig.6. The key insight of below heatmap is:

- Columns ‘energy’ and ‘loudness’ are strongly positively correlated which means louder songs tend to be more energetic.
- Columns ‘energy’ and ‘acousticness’ are strongly negatively correlated which means acoustic songs are usually quieter and calmer.
- Most other features are fairly independent, which is good for clustering.

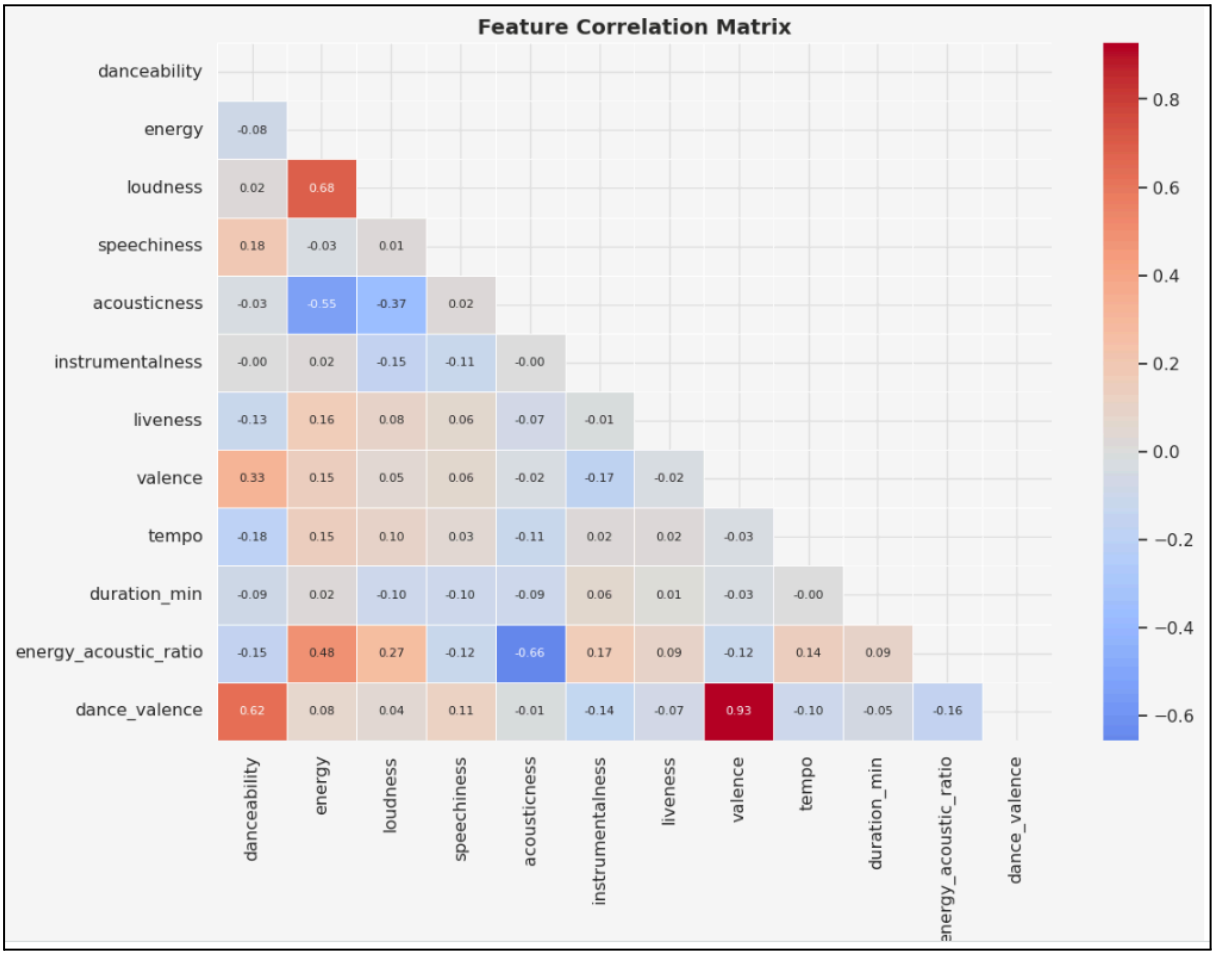


Fig.6. Feature Correlation Matrix.

3.3 The PCA Bar Chart

Before we're checking the most suitable dimension number, we need to standardise all features to the same scale, then compress them into fewer dimensions. The bar chart in Fig.7. shows how much variance one Principal Component (PC) explains the information. From this bar chart, we can see only 6 - 8 components are needed to explain 95% of the variance (out of 14 features), which means many original features overlap or carry similar information. PCA removes that redundancy, making the models faster and more accurate.

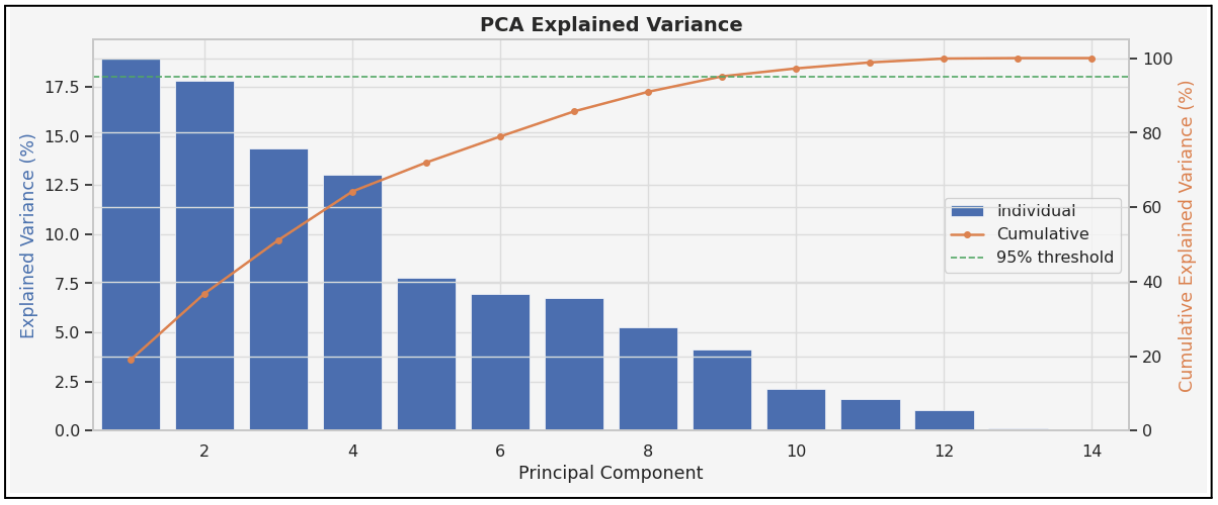


Fig.7.PCA Explained Variance.

Based on all the information above, we'll also use modern unsupervised learning methods like **The K-means, Gaussian Mixture Model (GMM) and Random Forest proximity matrices.** After running the models, we'll compare its evaluation matrix.

4. Implementing Classification and Model Evaluation

Before running all the models, we need to find the most optimal number of clusters with the Elbow Method in Fig.8. It turned out the most optimal number of clusters are in the range of 5. To ensure which n_cluster, we need to take a look at the Silhouette Score on Fig.9 where K=5 has silhouette core of ~0.16 which is the highest from the rest of the K value. We'll use K-5 on our K-means model.

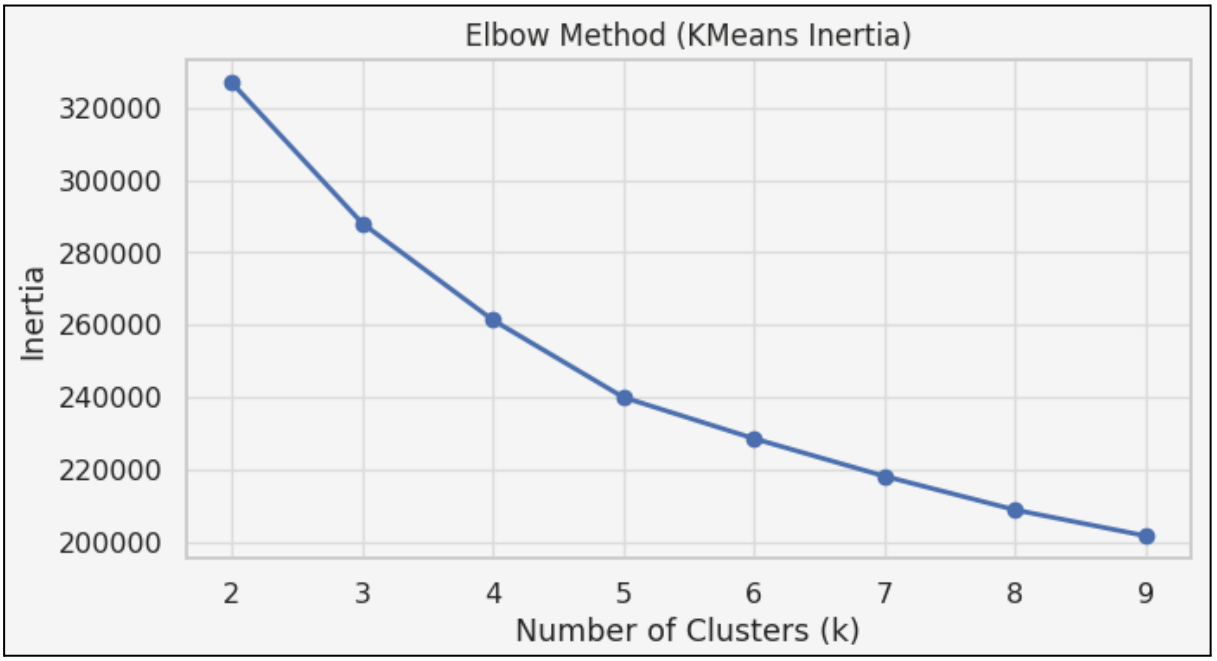


Fig.8. Elbow Method.

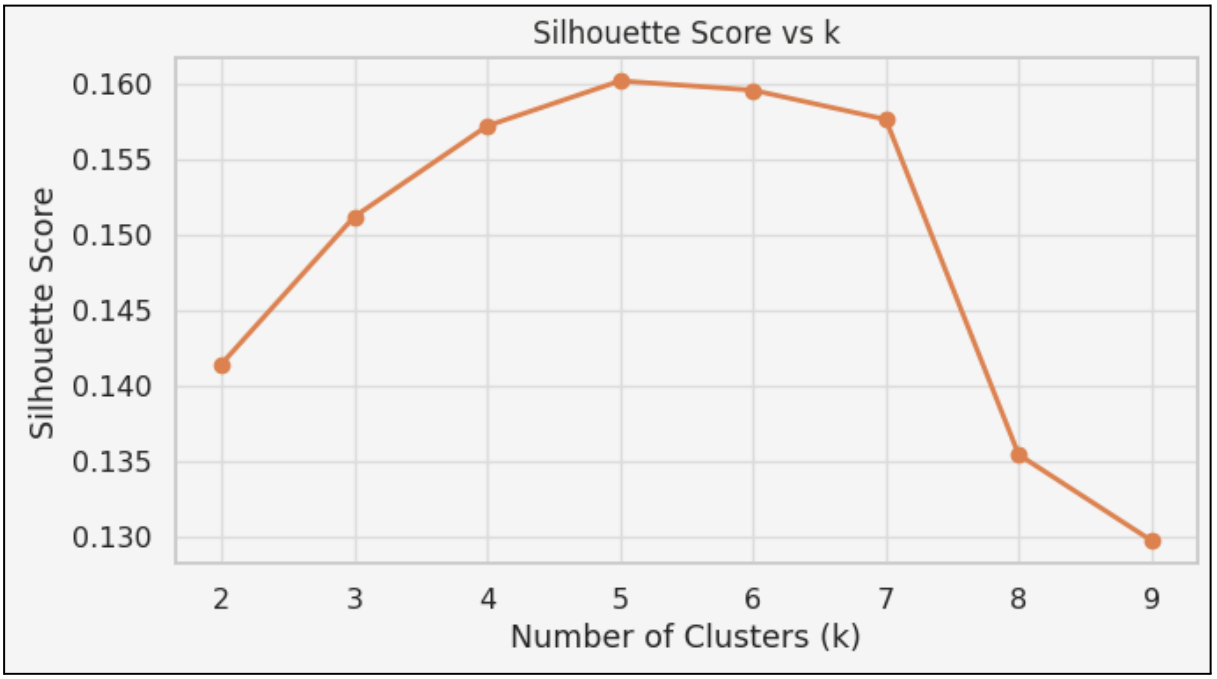


Fig.9. Silhouette Score.

After finding the most optimal k for K-means, we will run the decided clustering model and evaluate it. We'll go through the model side by side:

	Silhouette	Davies-Bouldin	Calinski-Harabasz
K-Means	0.1603	1.6376	4062.53
GMM	0.0441	3.3826	1475.29
RF+Hier	0.1570	1.6593	3966.13

Fig.9. Evaluation Matrix

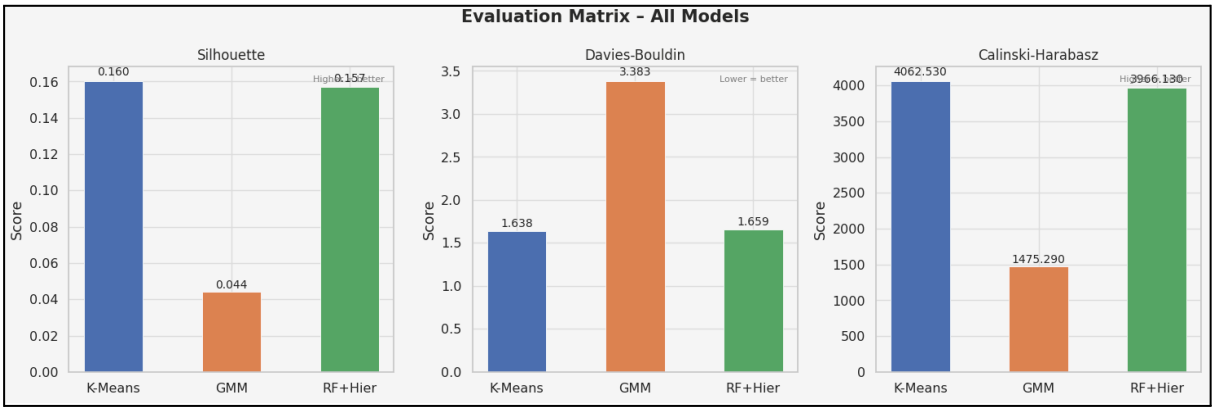


Fig.10. Histogram of Evaluation Matrix.

- **K-Means**

K-Means is the simplest of the three models. It works by repeatedly assigning each song to its nearest cluster center and adjusting those centers until they stabilize. Because of this, it assumes clusters are roughly spherical and equally sized which is rarely perfectly true in real music data. In the evaluation, K-Means typically scores decently on Calinski-Harabasz because it directly optimizes for tight, compact clusters, but it may underperform on Davies-Bouldin when clusters in the actual data are irregularly shaped or overlapping. In this dataset, K-means has the best score in Silhouette Score and Calinski-Harabasz. However, on the Davies-Bouldin score this model is in second place.

- **GMM (Gaussian Mixture Model)**

GMM is a more flexible upgrade over K-Means. Instead of hard assignments, it gives each song a probability of belonging to each cluster, then picks the most likely one. It also allows clusters to be oval/elliptical rather than perfectly round, making it better at handling groups that overlap in feature space which is common in music, where genres blend. GMM often achieves a better Silhouette and Davies-Bouldin score than K-Means because it doesn't force

rigid boundaries. However, it can be sensitive to initialization and may struggle if the data has very irregular, non-Gaussian shapes.

- **RF Proximity + Hierarchical Clustering**

  This is the most sophisticated of the three. A Random Forest is first trained using K-Means labels as pseudo-targets, then the proximity matrix is built by measuring for every pair of songs how often they land in the same leaf node across 150 trees. Songs that consistently co-occur in the same leaf are considered highly similar, regardless of their raw feature distances. This allows the model to capture non-linear, complex relationships that K-Means and GMM simply cannot see. The resulting distance matrix is then fed into Agglomerative Hierarchical Clustering, which builds clusters bottom-up by merging the most similar pairs first. This combination is the most computationally expensive, but it's also the most capable of finding clusters that genuinely reflect musical similarity rather than geometric proximity in feature space.

Based on the ranking of all three across Silhouette Score, Davies-Bouldin Index, and Calinski-Harabasz Index simultaneously, **then summing the ranks, The best model is K-means.** The other 2 models are more sophisticated than K-means, but the matrix evaluation showed that the dataset is not too complex. It's also shown in Fig.11. that only K-means model can create a cluster in the cluster visualisation below:

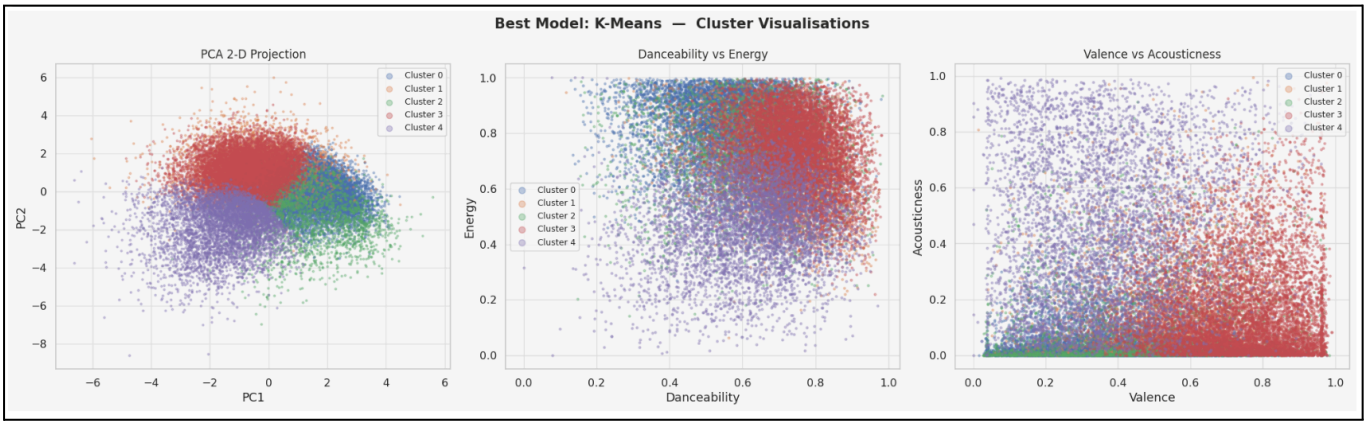


Fig.11. Models Cluster Visualisations.

5. Insight and Key Findings: Dataset Objective

In order to get a better understanding of the characteristic of each clustering group we need to extract the cluster mean profile and it tops the genre. That information can be found in Fig.12. To Fig.14. below:

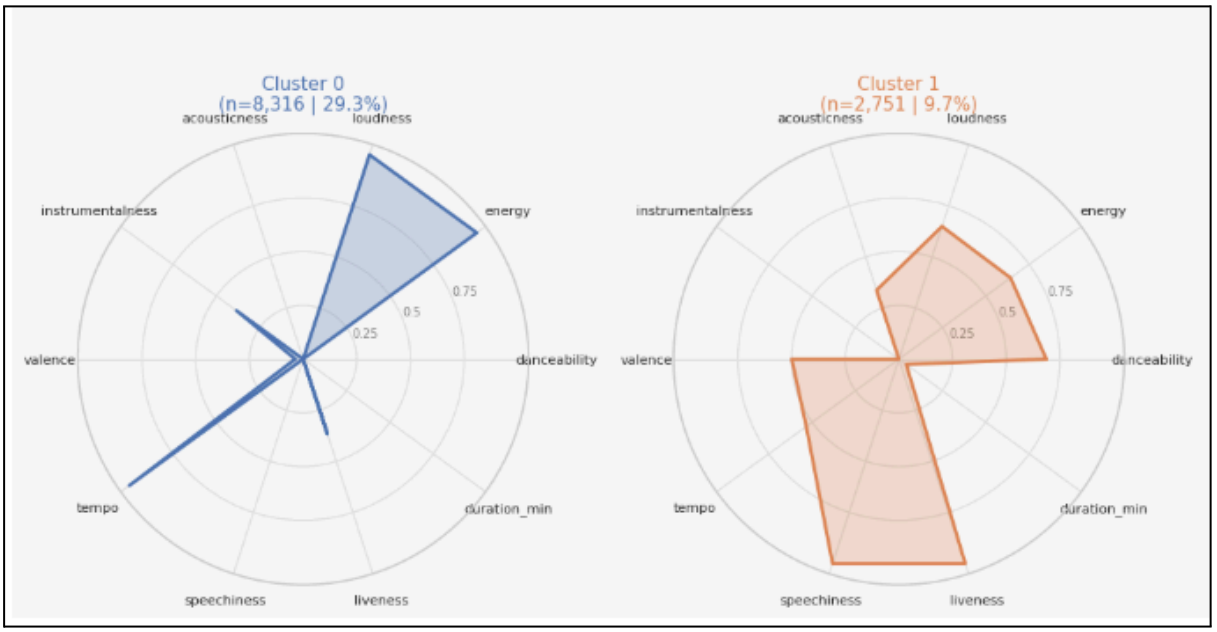
Cluster Mean Profiles:

cluster	danceability	energy	loudness	acousticness	instrumentalness	valence	tempo	speechiness	liveness	duration_min
0	0.570	0.816	-5.209	0.037	0.111	0.379	129.039	0.076	0.215	3.527
1	0.691	0.691	-6.783	0.178	0.024	0.542	122.038	0.326	0.322	3.627
2	0.633	0.707	-7.957	0.100	0.250	0.457	122.759	0.077	0.184	5.899
3	0.746	0.728	-6.259	0.163	0.034	0.719	116.904	0.096	0.155	3.584
4	0.614	0.467	-9.698	0.454	0.114	0.366	114.031	0.082	0.153	3.529

Fig.12. Cluster Mean Profiles.

```
Top genre per cluster:
  Cluster 0: edm   (32.7%)
  Cluster 1: rap   (53.8%)
  Cluster 2: edm   (27.3%)
  Cluster 3: latin   (25.3%)
  Cluster 4: r&b   (27.1%)
```

Fig.13.Top Genre Per Cluster.



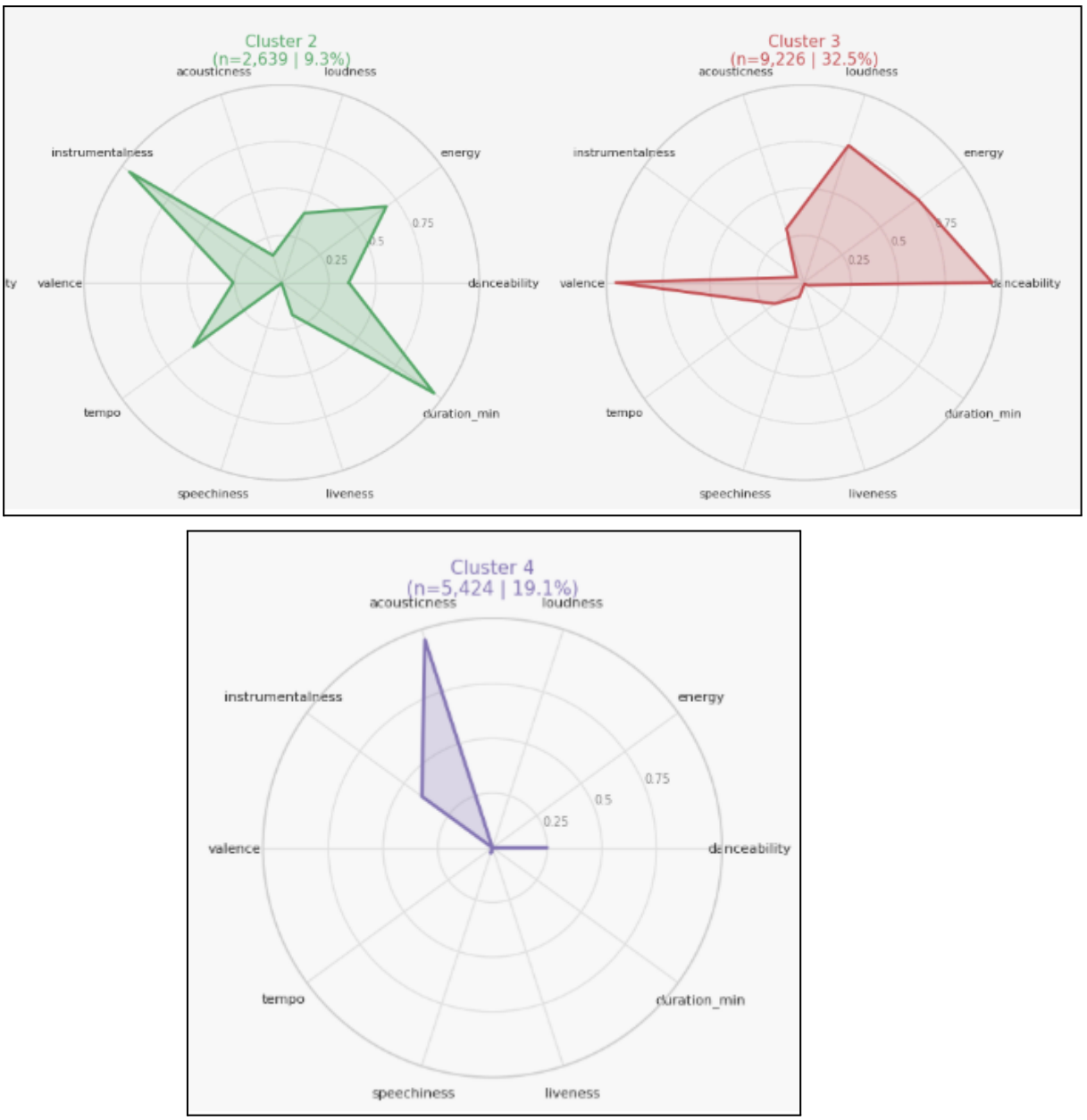


Fig.14. Cluster Mean Profiles.

The K-Means model successfully segmented the playlists not just by raw genre, but by core audio properties (vocal presence, instrumentalness, emotional valence, and energy). Below are the key findings and the specific listener profiles for each cluster:

1. **Cluster 0: The High-Octane Club Banger**
   - **Key Findings:** This cluster is defined by the **highest energy (0.816)** and **highest tempo (129 BPM)**. It has very low acousticness (0.037), meaning it is heavily produced and electronic.
   - **Dominant Genre:** EDM (32.7%)
   - **Target Listener:** Listeners seeking maximum stimulation. This is a "hype" music profile. It is highly suitable for gym playlists, long-distance running, or clubbing. These listeners are less concerned with deep lyrics; the primary focus is on a fast beat and a pumping bass line.

2. Cluster 1: The Flow & Rhyme

- **Key Findings:** The defining trait of this cluster is its massive speechiness (0.326), which is vastly higher than any other group. This indicates tracks dominated by spoken words rather than sung melodies. It also has strong danceability (0.691).
- **Dominant Genre:** Rap (53.8%)
- **Target Listener:** Aficionados of Hip-Hop and Rap culture. This listener engages with music for the vocal "flow" and rhymes of the artist. Because the instrumentals are usually stripped down to a basic beat (instrumentalness is very low at 0.024), the listener's attention is drawn squarely to the lyrics and delivery.

3. Cluster 2: The Extended Mix / Deep Trance

- **Key Findings:** This cluster has two massive outliers: **Duration (5.89 minutes)** and **Instrumentalness (0.250)**. While Cluster 0 represents short, punchy EDM, Cluster 2 represents long-form electronic music (like House, Techno, or Trance mixes) where vocals are sparse.
- **Dominant Genre:** EDM (27.3%)
- **Target Listener:** Deep-focus workers, programmers, or fans of underground electronic music. Because the tracks are long (averaging almost 6 minutes) and lyrics are minimal, this music serves as a non-distracting backdrop. It is perfectly suited for deep work playlists, coding sessions, or simply rhythmic background music.

4. Cluster 3: The Party Starter

- **Key Findings:** This is the "happiest" cluster. It holds the **highest valence (0.719)** which means the tracks sound positive, cheerful, and euphoric. It also boasts the **highest danceability (0.746)**.
- **Dominant Genre:** Latin (25.3%)
- **Target Listener:** People who are looking for a mood booster. The music in this cluster is great for casual dancing, playing during a road trip with friends, or as the soundtrack to a BBQ party. The vibe is cheerful, rhythmic, and very accessible to mainstream audiences (pop-friendly).

5. Cluster 4: The Unplugged & Chill

- **Key Findings:** A complete contrast to Cluster 0. This group is defined by the **lowest energy (0.467)**, **lowest loudness (-9.698 dB)**, and the **highest acousticness (0.454)**. The tempo is also the slowest (114 BPM).
- **Dominant Genre:** R&B (27.1%)
- **Target Listener:** People who need "healing" or tranquility. This music is dominated by acoustic instruments (like acoustic guitar or slow piano) combined with soft vocals. It is highly recommended for late-night drive playlists, relaxed studying at a cafe, unwinding in the afternoon, or as a bedtime playlist.

6. Model Plan of Improvement

Based on the overlap clustering in Fig.11 it indicates that standard K-Means is struggling with the continuous, blob-like distribution of the audio features. To improve this, we should refine the feature space (via scaling, selection, or UMAP), re-evaluate the optimal number of clusters, and test probabilistic or density-based algorithms like Gaussian Mixture Models (GMM) or HDBSCAN that handle overlapping data much better than K-Means.